Akademia WSB

Dąbrowa Górnicza, Cieszyn, Olkusz, Żywiec, Kraków

WSB University

Projekt Zespołowy – Chmura Obliczeniowa

Zanim zaczniemy

1. Prowadzący: Oskar Zera
2. Kontakt: oskar.zera2@wsb.edu.pl
3. Zajęcia: zgodnie z harmonogramem dziekanatu

Warunki zaliczenia przedmiotu

- Obecność sprawdzana jest w ciągu pierwszych 15 minut zajęć.
- Zajęcia (ćwiczenia i laboratoria) są obowiązkowe.
- Nieobecność może skutkować brakiem możliwości zaliczenia przedmiotu.
- **Warunkiem zaliczenia jest realizacja projektu grupowego z wykorzystaniem technologii chmurowych.**
- **Na ostatnich zajęciach każda grupa będzie prezentowała działającą aplikację oraz raport opisujący projekt.**

Cel Projektu

W ramach laboratoriów zespoły przygotują i zrealizują aplikację działającą w środowisku chmurowym.

Projekt powinien:

- **działać online**
- posiadać interfejs użytkownika
- posiadać 4–6 głównych funkcjonalności
- korzystać z bazy danych
- posiadać podstawową autoryzację użytkowników
- zostać zaplanowany i wykonany zespołowo

Grupy: 3 – 4 osób

Minimalne wymagania projektu

Minimalne wymagania dotyczą struktury aplikacji, czyli tego **z czego system musi się składać technicznie**.

Projekt musi zawierać:

- aplikację webową
- bazę danych
- system logowania użytkownika
- działające wdrożenie w chmurze

Każda aplikacja powinna mieć **4–6 głównych funkcjonalności**, które tworzą minimalną działającą wersję systemu, czyli tzw. MVP — Minimum Viable Product.

Kryteria oceny projektu

1. **Kompleksowość projektu**
   zgodność z założeniami funkcjonalnymi i technicznymi
2. **Poprawność działania**
   stabilność aplikacji, poprawne działanie funkcjonalności
3. **Wykorzystanie technologii chmurowych**
   zastosowanie usług chmurowych oraz ich poprawna konfiguracja
4. **Dokumentacja projektu**
   raport opisujący projekt, architekturę systemu i sposób wdrożenia

Ocena końcowa obejmuje:

- działającą aplikację
- wdrożenie w chmurze
- raport projektowy

Najważniejsza zasada projektu

Projekt powinien być **realny do ukończenia**.

Najczęstszy problem projektów to:

- zbyt duży zakres
- zbyt skomplikowana architektura
- brak planu pracy

Dlatego:

lepiej stworzyć prostą aplikację, która działa, niż ambitny projekt, którego nie uda się ukończyć.

Jak wybrać dobry projekt

Dobry projekt powinien:

- mieć jasno określonego głównego użytkownika
- posiadać 4–6 głównych funkcjonalności
- mieć prosty model danych
- być możliwy do ukończenia w semestrze (25h)

Przykłady odpowiednich projektów:

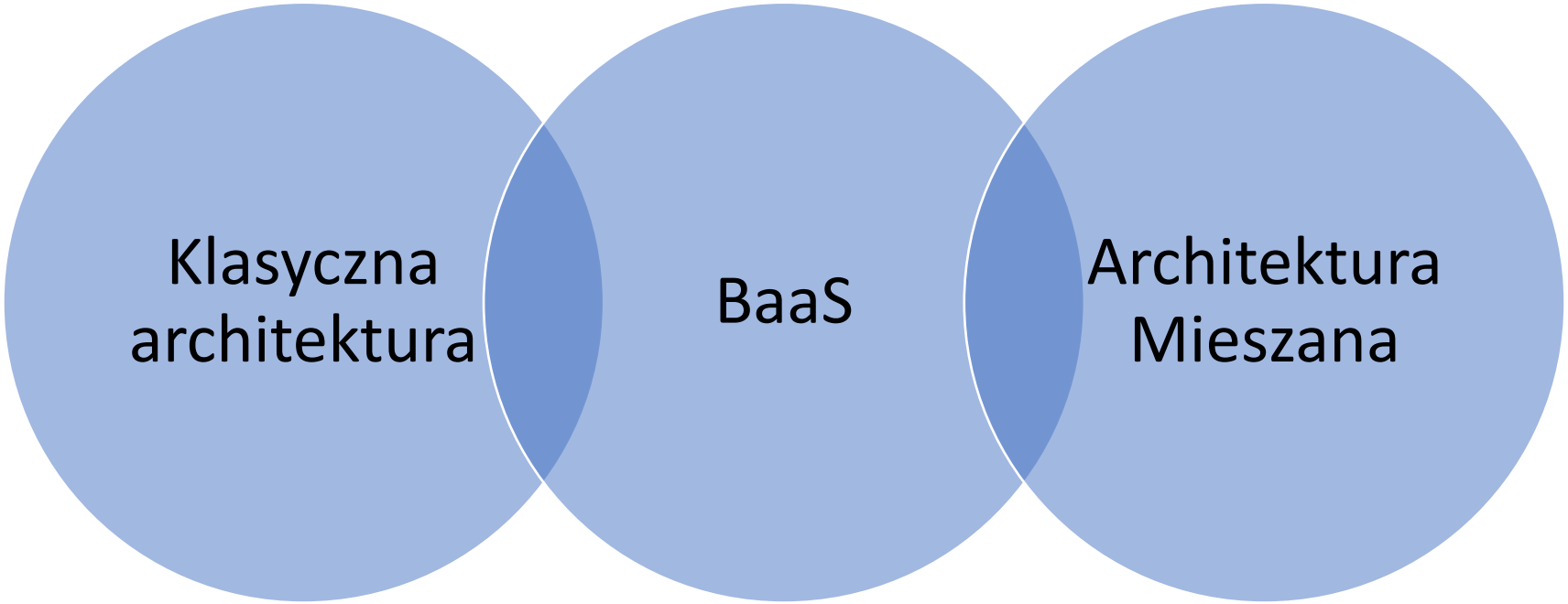
- aplikacja do zarządzania zadaniami
- system wydarzeń
- system rezerwacji
- system notatek

Cel dzisiejszych zajęć

Podczas dzisiejszych laboratoriów każda grupa powinna przygotować:

- pomysł projektu
- listę funkcjonalności MVP(Most Viable Product)
- wstępną architekturę systemu
- model danych
- podział ról w zespole

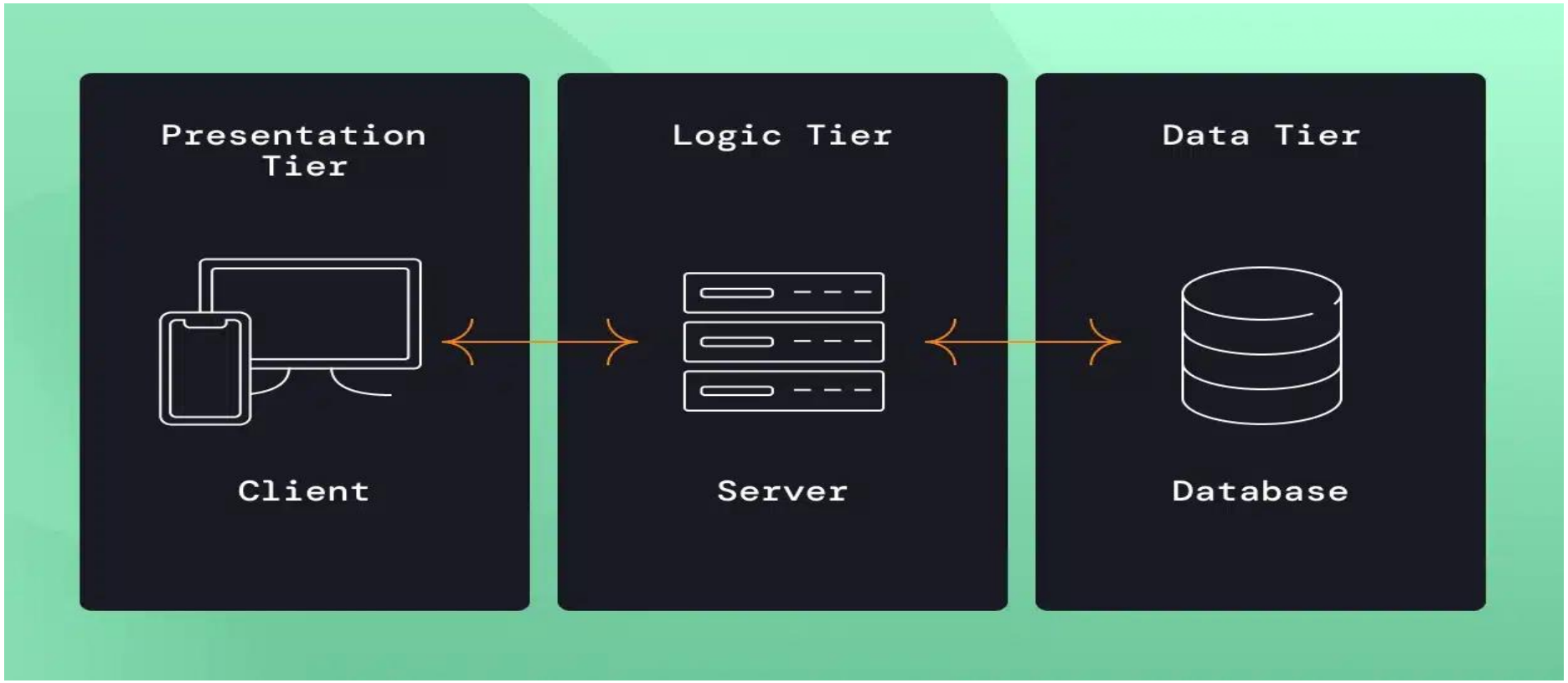
Którą architekturę wybrać?



Niezależnie od wybranej architektury projekt powinien zawierać:

- interfejs użytkownika
- logikę aplikacji
- warstwę danych
- wdrożenie w chmurze

Klasyczna Architektura Trójwarstwowa



Architektura oparta o BaaS

Frontend

- Next.js
- React
- Vue
- Angular

Supabase (usługa backendowa Baas)

- Auth
- Baza Danych
- Zasoby
- API

Architektura mieszana

Frontend
- Next.js
- React
- Vue
- Angular

Backend/API
- Java
- Node
- Go
- C#

Usługa backendowa (BaaS)
- Baza danych
- Auth
- Storage

Dzisiejsze zadanie - draft koncepcji projektu chmurowego

Każdy zespół przygotowuje krótki dokument zawierający:

- tytuł projektu i skład zespołu (imie, nazwisko, numer albumu)
- opis koncepcji systemu
- funkcjonalności MVP
- wybór technologii chmurowej z uzasadnieniem
- wstępny szkic architektury
- podział ról w zespole

Forma oddania: zadanie na Moodle

Termin: przed zakończeniem zajęć

Jak będzie realizowany projekt

Wymagania dotyczące raportu koncepcji

- Każdy zespół przygotowuje raport w formacie **PDF** o maksymalnej objętości **2 stron**. Dokument należy przesłać w zadaniu na platformie **Moodle** przed zakończeniem zajęć. Każdy członek zespołu powinien przesłać **ten sam plik raportu** we własnym imieniu.
- Raport powinien być **spójny, czytelny oraz przygotowany w sposób profesjonalny**, ponieważ będzie stanowił podstawę do **akceptacji tematu przez prowadzącego** oraz punkt wyjścia do dalszej realizacji projektu.

Zakres projektu – etap I

Analiza wymagań i projekt architektury

Na pierwszym etapie projektu zespół powinien przeprowadzić analizę wymagań systemu oraz zaprojektować wstępną architekturę aplikacji.

Analiza wymagań obejmuje określenie podstawowych funkcjonalności systemu, takich jak na przykład:

- tworzenie i zarządzanie wydarzeniami
- rejestracja uczestników
- wysyłanie powiadomień lub przypomnień
- obsługa płatności za bilety (opcjonalnie)

Na tym etapie należy również dokonać wstępnej analizy dostępnych technologii chmurowych (np. AWS, Microsoft Azure, Google Cloud, Firebase, Supabase lub inne rozwiązania).

Projekt architektury systemu powinien uwzględniać główne komponenty aplikacji, takie jak:

- warstwa interfejsu użytkownika (frontend)
- logika aplikacji (backend lub usługi platformowe)
- baza danych
- usługi chmurowe i mechanizmy bezpieczeństwa

Zakres projektu – etap II

Implementacja, testowanie i wdrożenie

Na etapie implementacji zespół powinien przygotować działającą aplikację składającą się z kilku podstawowych komponentów.

- System powinien obejmować:
- interfejs użytkownika (frontend)
- logikę aplikacji (backend lub usługi platformowe)
- bazę danych przechowującą dane systemu
- API umożliwiające komunikację pomiędzy komponentami

W projekcie należy również przeprowadzić podstawowe testy działania systemu, w szczególności:

- testy funkcjonalne
- podstawowe testy wydajnościowe

Po zakończeniu implementacji aplikacja powinna zostać wdrożona w środowisku chmurowym wraz z podstawową konfiguracją monitorowania działania systemu.

Projekt kończy się przygotowaniem raportu końcowego oraz demonstracją działającej aplikacji.

Zakres projektu – etap III

Technologie i organizacja pracy

W projekcie można wykorzystać różne technologie i narzędzia, w zależności od charakteru systemu.

- **Frontend**: React, Angular, Vue.js lub inne frameworki frontendowe
- **Backend**: Node.js, Python, Java, PHP lub rozwiązania serverless
- **Backend as a Service**: Supabase, Firebase
- **Bazy danych**: PostgreSQL, MySQL, MongoDB, Firestore

Zarządzanie projektem

Każdy zespół powinien prowadzić **board projektu**, który umożliwia planowanie i śledzenie postępów pracy.

Do zarządzania zadaniami można wykorzystać narzędzia takie jak:

- GitHub Projects/Trello/Jira/Notion

Minimalna dokumentacja projektu

W projekcie wymagana jest podstawowa dokumentacja:

1. **Diagram przypadków użycia - link**
   pokazuje użytkowników i funkcjonalności
2. **Diagram architektury systemu - link**
   pokazuje główne komponenty aplikacji
3. **Model danych – link**
   pokazuje encje i relacje

Role w zespole

Koordynator / Tech Lead

- pilnuje kierunku projektu
- dba o spójność architektury
- koordynuje pracę zespołu

Frontend Developer

- odpowiada za interfejs użytkownika
- integruje frontend z danymi i API

Backend / Data Developer

- odpowiada za logikę systemu, bazę danych, autoryzację

DevOps / QA

- odpowiada za wdrożenie, konfigurację środowiska, testy i jakość projektu

Uwaga: role nie są sztywne i członkowie zespołu mogą łączyć kilka obszarów.

Zarządzanie projektem

Przykładowy workflow:

Backlog → To Do → In Progress → Review → Done

Board powinien zawierać:

- listę zadań
- przypisanie osób
- aktualny status pracy

Pomoce

- Hostingi: https://github.com/iSoumyaDev/Awesome-Web-Hosting-2026
- Roadmaps: https://roadmap.sh
- Darmowe tiery na cloud: https://github.com/cloudcommunity/Cloud-Free-Tier-Comparison
- Pomysły: https://dev.to/kishansheth/200-project-ideas-from-beginner-to-advanced-with-open-source-contributions-3g6a